Proposta para a equipe - Fase 4 na AWS

Projeto: NumberOne. **Equipe:** Matheus, Julio, Anderson e Marcelo. **Prazo de planejamento:** três semanas. **Data:** 27/10/2026.

Guia de leitura

- Comece pelo dicionário: os termos usados pela equipe estão reunidos antes da explicação do processo.
- As seções 1 e 2 definem o escopo de três semanas, os serviços e os componentes AWS.
- As seções 3 a 6 explicam antes/depois, reserva, compensação, SQS e transações por banco.
- A seção 7 organiza as três semanas; a seção 8 reúne o checklist completo.
- A seção 10 apresenta as entregas de Matheus, Julio, Anderson e Marcelo.

Leitura técnica e planejamento da equipe

Os exemplos descrevem a implementação proposta para a próxima etapa. O checklist permite acompanhar a construção e reunir as evidências da entrega.

Dicionário para começar a leitura

Termo da Saga	Aplicação neste projeto
Saga / orquestrador	Processo com etapas locais; código no serviço de ordem salva o andamento e decide o próximo comando
Estado da Saga	Etapa técnica, como aguardando pagamento ou compensando; diferente do status comercial da ordem
Comando	Pedido dirigido a um serviço: `StartDiagnosis` solicita diagnóstico
Evento	Fato ocorrido: `DiagnosisCompleted` informa que o diagnóstico terminou
Compensação	Nova operação para neutralizar um efeito anterior: liberar reserva ou estornar pagamento
Outbox	Mensagem pendente salva junto do efeito de negócio; um worker publica no SQS
Inbox / idempotência	Registro e regras que evitam repetir efeitos de mensagens reprocessadas
Correlação / causalidade	`sagaId` identifica o processo; `causationId` aponta a mensagem que provocou a ação
Timeout / reconciliação	Prazo vencido deixa o resultado incerto; conferir participante/provedor antes de decidir
Retentativa / DLQ	Recuperar falhas transitórias; guardar mensagens esgotadas para análise e reprocessamento

1. Objetivo e escopo das três semanas

Entregar três microsserviços, um atendimento completo, Mercado Pago, Saga com compensação e deploy independente em Kubernetes. Usaremos **SQS para comunicação** e **DynamoDB para execução e estoque**, reduzindo operação de infraestrutura.

O escopo inicial terá uma revisão aprovada e uma tentativa financeira ativa por atendimento, reserva simples e prazo global coordenado pela Saga. Novas revisões após aprovação e reinício financeiro de processos encerrados ficam para evolução. Manter identificação de filial, sem construir gestão completa de filiais.

Reaproveitar Java 25/Spring Boot 4, Maven, módulos, regras e testes atuais. Manter features com API/aplicação/domínio/infraestrutura, DTOs separados, gateways e injeção por construtor. Os recursos AWS são uma proposta; o checkout consultado não comprova provisionamento. Recuperar a base da fase 3 que o grupo confirmar.

2. Serviços, bancos e componentes AWS

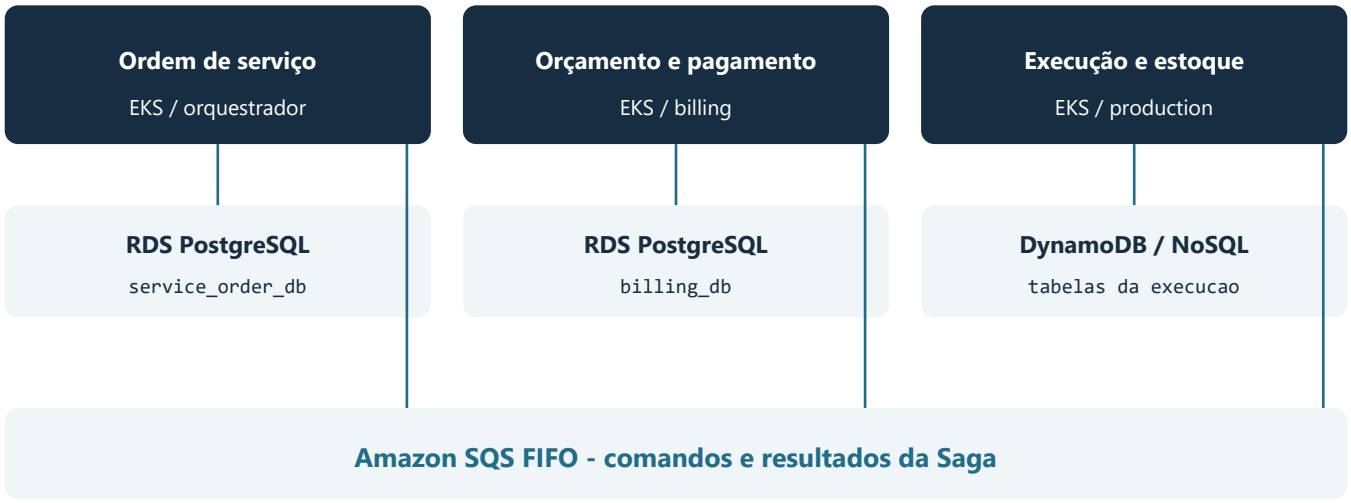
Serviço / responsável	Repositório proposto	Persistência exclusiva
Ordem e orquestrador / **Matheus**	`numberone-service-order`	**RDS PostgreSQL** `service_order_db`: clientes, veículos, ordem, histórico, Saga e inbox/outbox
Orçamento e pagamento / **Julio**	`numberone-billing`	**RDS PostgreSQL** `billing_db`: orçamento, pagamento, notificações, operações externas e inbox/outbox
Execução e estoque / **Anderson**	`numberone-production`	**DynamoDB**: tabelas de trabalho, catálogo, saldo, reserva, movimentos e inbox/outbox

Marcelo lidera plataforma e publica infraestrutura/guias em `numberone-platform`. Cada serviço mantém suas próprias imagens, pipeline e recursos; acessa somente seus dados. O estado da Saga continua no PostgreSQL da ordem. Tabelas DynamoDB de execução não são compartilhadas com billing ou ordem.

Componente AWS	Uso proposto
EKS	Executar as três aplicações em Kubernetes; reaproveitar o cluster da fase 3 quando disponível
ECR	Repositório de imagens por aplicação, tag do commit e digest
RDS PostgreSQL	Bancos de ordem/billing com credenciais, migrations e acesso separados; IaC prevê recursos próprios por serviço
DynamoDB	NoSQL de negócio de execução, com escrita condicional e `TransactWriteItems`
SQS FIFO + DLQs	Três filas principais e uma DLQ FIFO para cada uma
IAM / EKS Pod Identity	Permissões por ServiceAccount para filas, tabelas e secrets; sem chaves AWS fixas nos containers
Secrets Manager	Credenciais SQL e segredos Mercado Pago; documentar o carregamento na aplicação
ALB + ACM	Entrada HTTPS das APIs/webhook, usando Ingress/controller compatível com o ambiente
CloudWatch	Logs e alarmes; integrar tracing e dashboards já entregues na fase 3

CI/CD proposto: GitHub Actions com OIDC, ECR e deploy EKS. SonarQube continua no CI. O e-mail usa o provedor funcional do projeto; SES é opcional se o acesso de envio já estiver pronto.

Para o prazo, manter o orquestrador Java e o publicador de outbox na aplicação. Step Functions, SNS, EventBridge, Lambda e DynamoDB Streams ficam como opções futuras, fora do caminho crítico desta entrega.



Billing integra Mercado Pago por HTTPS e recebe webhook.

3. Como era e como ficará

Exemplo: troca de pastilhas, **R$ 150 de mão de obra + um kit de R$ 200 = R$ 350**. Estoque inicial de cinco kits. `os-123` é um nome fictício; usar UUID nos contratos.

Operação	Hoje: mesma aplicação/PostgreSQL	Nova divisão
Abrir e acompanhar	`ServiceOrderService`, `customer` e `vehicle`	Ordem mantém atendimento e Saga no seu PostgreSQL
Diagnóstico, item e insumo	Dentro de `serviceorder`	Execução cria trabalho e diagnóstico no DynamoDB
Gerar/aprovar orçamento	`ServiceOrderBudgetService` altera diretamente a ordem	Billing salva no próprio PostgreSQL e envia eventos
Consultar/baixar estoque	`inventory`; baixa ao iniciar item	Execução reserva antes da cobrança e consome ao iniciar item
Iniciar trabalho	Inicia ordem e depois item	Ordem autoriza após pagamento; equipe inicia pela API de execução
Pagamento	Integração não identificada	Billing cria checkout, valida confirmação e estorna quando necessário
Concluir e entregar	Operações no mesmo banco	Execução informa conclusão; ordem registra acompanhamento/entrega

Hoje, iniciar a ordem não baixa estoque. `startServiceOrderItem` verifica/consome insumos ao iniciar o item e pode marcar espera por peças. Reserva antecipada, pagamento e Saga são acréscimos. Ver exemplo com endpoints atuais.

3.1 HTTP, SQS e chamadas locais

`StartDiagnosis` **e** `CreateBudget` **são mensagens SQS, não endpoints REST.** A equipe abre a ordem por HTTP e recebe confirmação local; não espera a Saga inteira. Registra diagnóstico pela API de execução. O cliente aprova pela API de billing. Billing usa HTTPS para Mercado Pago e recebe webhook.

Dentro de execução, verificar estoque é chamada Java local:

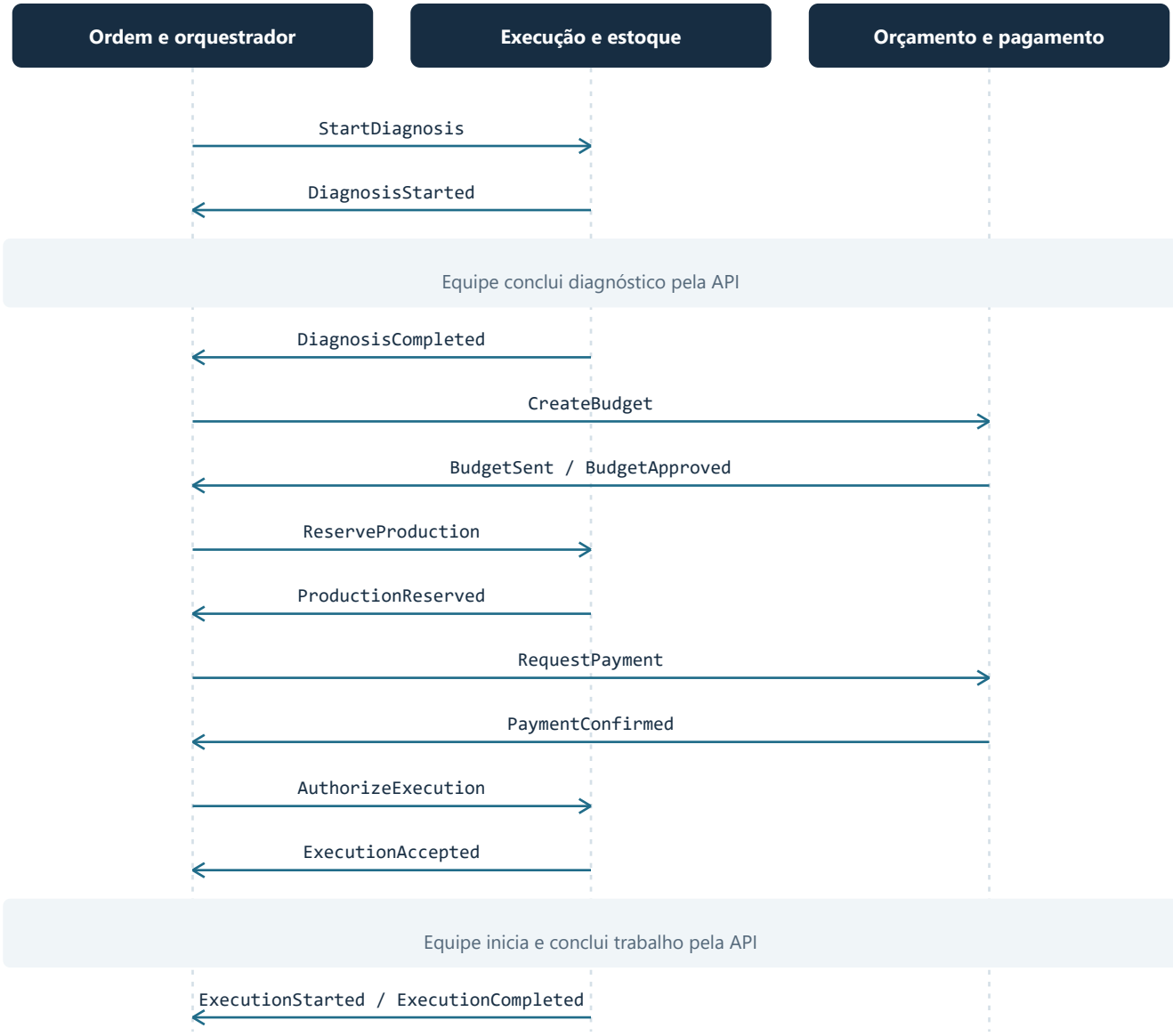
```
HTTP -> ExecutionController.startItem(id)
     -> ExecutionService.startItem(id)
     -> ReservationService.consume(...)
     -> DynamoDB: saldo + reserva + movimento + trabalho + outbox
     -> resposta HTTP após sucesso da transação
```

As chamadas locais montam uma unidade de escrita. No PostgreSQL, usar transação local. No DynamoDB, o adaptador usa explicitamente `TransactWriteItems`: `@Transactional` sozinho não torna várias chamadas SDK uma transação.

3.2 Sequência principal

1. Ordem grava atendimento, Saga e `StartDiagnosis` na outbox PostgreSQL; o worker publica na fila de execução.
2. Execução cria trabalho no DynamoDB e envia `DiagnosisStarted`. Ordem recebe e atualiza acompanhamento.
3. A equipe conclui o diagnóstico pela API de execução. Dados e `DiagnosisCompleted` são gravados juntos no DynamoDB; é uma notificação técnica para a ordem.
4. Orquestrador recebe o evento e grava `CreateBudget` na outbox PostgreSQL. Billing recebe composição/destinatário e cria orçamento, sem ler DynamoDB.
5. Billing envia orçamento e publica `BudgetSent`; a aprovação HTTP do cliente gera `BudgetApproved`.
6. Ordem pede `ReserveProduction`; execução reserva um kit e responde `ProductionReserved`.
7. Ordem pede `RequestPayment`; billing confirma o pagamento com Mercado Pago e publica `PaymentConfirmed`.

8. Ordem pede `AuthorizeExecution`; execução aceita e envia `ExecutionAccepted`. A Saga de liberação termina, deixando a ordem pronta para execução.
9. Equipe inicia o item pela API; execução consome a reserva e envia `ExecutionStarted`. Ao concluir trabalho, publica `ExecutionCompleted`. Ordem atualiza os estados e recebe a entrega por HTTP.



As setas entre serviços são SQS. Eventos separados por `/` são publicações distintas em momentos diferentes. Não existe HTTP aberto esperando cliente ou mecânico.

4. Reserva e compensação simplificadas

4.1 Reservar, consumir ou liberar

A reserva identifica Saga/tentativa, trabalho, filial e itens aprovados. Uma transação DynamoDB valida todos os insumos, atualiza saldos e grava reserva, trabalho, inbox/outbox. Todos os itens são reservados ou nenhum. Chave de operação e escritas condicionais impedem duplicação.

Operação	**Físico** `onHand`	**Reservado** `reserved`	**Disponível** `available`
Inicial	5	0	5

Operação	Físico onHand	Reservado reserved	Disponível available
Reservar um kit	5	1	4
Consumir ao iniciar item	4	0	4
Ou liberar antes do consumo	5	0	5

Manter `available = onHand - reserved`. Estados da reserva: `RESERVED`, `CONSUMED`, `RELEASED`; controlar por item nos trabalhos maiores. Autorização é estado do trabalho. A Saga controla o prazo de pagamento e a limpeza; **visibility timeout do SQS e TTL DynamoDB não são o prazo de negócio**.

Autorizar, iniciar e cancelar validam/atualizam a versão do trabalho na mesma transação dos demais efeitos. Comando atrasado não reativa cancelamento. Reserva autorizada permanece até consumo/cancelamento confirmado. Ajuste manual não pode retirar quantidade comprometida.

4.2 Falhas do escopo inicial

Situação	Tratamento e persistência
Orçamento rejeitado	Billing salva rejeição no PostgreSQL; ordem cancela trabalho e encerra com resultado `REJEITADA`
Falta estoque	DynamoDB não confirma efeitos parciais; execução registra recusa e a ordem não pede pagamento
Pagamento recusado e tentativa encerrada	Billing informa falha; ordem grava `COMPENSATING`; execução libera reserva no DynamoDB e billing encerra a tentativa no PostgreSQL
Pagamento aprovado e trabalho cancelado antes do início	Confirmar cancelamento operacional; billing prepara estorno no PostgreSQL, chama Mercado Pago e confirma resultado antes do encerramento
Timeout financeiro	Billing concilia com o provedor; ordem mantém a pendência no PostgreSQL
Reparo já começou	Registrar intervenção operacional; não repor automaticamente peça consumida

Saga só fica `COMPENSATED` após `ProductionCancelled` e `BillingCompensated`. Estorno solicitado não significa concluído. Falhas técnicas têm retentativa; pendências esgotadas geram alerta e intervenção, com histórico preservado.

5. SQS e exemplos de implementação

5.1 Filas e contratos

Fila FIFO	Consumidor	Mensagens
`numberone-production-commands.fifo`	Execução	`StartDiagnosis`, `ReserveProduction`, `AuthorizeExecution`, `CancelProduction`
`numberone-billing-commands.fifo`	Billing	`CreateBudget`, `RequestPayment`, `CancelBilling`
`numberone-order-results.fifo`	Ordem	Resultados de diagnóstico, orçamento, reserva, pagamento, execução e compensação

Cada fila tem uma DLQ FIFO. Publicar com `MessageGroupId = sagaId` e `MessageDeduplicationId = messageId`. A ordenação vale por grupo na fila, não globalmente entre filas. Idempotência e validação de estado continuam obrigatórias.

O publicador escolhe a URL da fila. O consumidor lê o tipo no envelope e despacha para handler local. Envelope v1: IDs de mensagem/Saga/ordem/filial/tentativa, tipo, versão do schema, revisão/versão, causalidade, horário UTC e payload. Reenvio preserva ID; uma nova resposta cria ID novo e mantém correlação.

5.2 Inbox, outbox e confirmação

Nos SQLs, gravar negócio + inbox/outbox na mesma transação. Na execução, incluir esses itens em `TransactWriteItems`. Publicar no SQS depois, por worker da aplicação; não adicionar Streams/Lambda ao fluxo inicial.

O worker assume lote com prazo de processamento, envia e marca publicação após sucesso. Em queda pode reenviar com o mesmo ID. Outbox DynamoDB terá índice para pendentes; não fazer varredura completa recorrente. Preservar a sequência pendente da mesma Saga.

O consumidor só chama `DeleteMessage` após commit local. Em erro, deixa reentregar e, após limite, encaminhar à DLQ. Ajustar visibility timeout ao processamento, com extensão quando necessário. Apagar o comando após registrar sua aceitação; não deixá-lo em processamento durante a espera humana.

5.3 Código Java de referência

Recortes com AWS SDK for Java v2 e Spring; construtores, imports e adaptadores seguem o projeto. Não são código já implementado. `codec` trata o envelope v1 e os records públicos ficam em arquivos próprios.

```
public record StartDiagnosis(
        UUID messageId, UUID sagaId, UUID serviceOrderId) {}

public record DiagnosisCompleted(
        UUID messageId, UUID sagaId, UUID serviceOrderId,
        List<QuotedItem> items) {}
```

Orquestrador, usando somente o PostgreSQL da ordem:

```
@Transactional
public void onDiagnosisCompleted(DiagnosisCompleted event) {
    if (!inbox.registerIfAbsent("diagnosis", event.messageId())) {
        return;
    }
    Saga saga = sagaGateway.findForUpdate(event.sagaId());
    saga.waitForBudget(event.serviceOrderId());
    sagaGateway.save(saga);
    outbox.append(CreateBudget.from(event));
}
```

Publicador SQS, fora da transação de negócio:

```
public void publish(OutboxMessage pending) {
    sqs.sendMessage(SendMessageRequest.builder()
            .queueUrl(pending.queueUrl())
            .messageGroupId(pending.sagaId().toString())
            .messageDeduplicationId(pending.id().toString())
            .messageBody(pending.json())
            .build());
    outbox.markPublished(pending.id());
}
```

Consumidor com long polling e exclusão após a regra local confirmar:

```
public void pollOnce() {
    ReceiveMessageResponse batch = sqs.receiveMessage(r -> r
            .queueUrl(queueUrl).waitTimeSeconds(20)
            .maxNumberOfMessages(1));
    for (Message message : batch.messages()) {
        handler.handle(codec.read(message.body()));
        sqs.deleteMessage(r -> r.queueUrl(queueUrl)
                .receiptHandle(message.receiptHandle()));
    }
}
```

O loop supervisionado continua após erros e não apaga a mensagem com falha. O SDK usa região e credenciais da role do pod. O exemplo processa uma mensagem por vez para facilitar ordenação.

Na compensação, Matheus recebe `PaymentFailed`, muda a Saga e grava `CancelProduction`. Anderson libera reservas ativas uma única vez e grava `ProductionCancelled` na transação DynamoDB. Depois Matheus pede `CancelBilling`; Julio confirma apenas após encerrar os efeitos financeiros. O anexo contém a escrita condicional de reserva e os detalhes de persistência.

6. Transações e desenvolvimento em paralelo

Operação local	Banco	Próxima mensagem
Abrir ordem e iniciar Saga	PostgreSQL da ordem	`StartDiagnosis`
Criar trabalho; concluir diagnóstico	DynamoDB, em etapas separadas	`DiagnosisStarted`; depois `DiagnosisCompleted`
Receber diagnóstico	PostgreSQL da ordem	`CreateBudget`
Gerar/enviar/aprovar orçamento	PostgreSQL de billing, por etapa	`BudgetSent`; depois aprovação/rejeição
Receber aprovação	PostgreSQL da ordem	`ReserveProduction`
Reservar todos os insumos	DynamoDB, uma transação	Reserva confirmada ou recusada
Receber reserva	PostgreSQL da ordem	`RequestPayment`
Preparar checkout e conferir pagamento	PostgreSQL de billing; HTTP externo entre transações	Pagamento pendente, confirmado ou falhou
Receber pagamento confirmado	PostgreSQL da ordem	`AuthorizeExecution`
Autorizar trabalho	DynamoDB	`ExecutionAccepted`
Receber autorização/início/conclusão	PostgreSQL da ordem, por evento	Estados de pronto, execução e finalizado
Consumir e iniciar item; concluir trabalho	DynamoDB, por operação	`ExecutionStarted`; depois `ExecutionCompleted`
Cancelar trabalho/reserva	DynamoDB	`ProductionCancelled` ou `ExecutionAlreadyStarted`
Encerrar cobrança/confirmar estorno	PostgreSQL de billing; provedor fora da transação	`BillingCompensated`
Receber compensações/registrar entrega	PostgreSQL da ordem, por operação	Histórico/estado final

Essas linhas não representam uma transação distribuída única. O anexo técnico enumera T01-T26 e as compensações, com cada banco. Toda recepção atualiza inbox e andamento local. A Saga de liberação termina na autorização; reparo e entrega continuam por eventos.

Contrato v1 nos primeiros dias: Marcelo publica filas/IAM, envelope e aplicação mínima. Donos definem schemas e amostras de negócio, incluindo erros. `CreateBudget` leva composição e destinatário; reserva/autorização/cancelamento compartilham tentativa/trabalho. Mudanças incompatíveis exigem versão nova.

Matheus testa resultados e comandos com fixtures. Julio testa comandos e provedor simulado. Anderson testa reservas/consumo no DynamoDB. Marcelo valida infraestrutura e pipelines com aplicação mínima. A integração substitui fixtures pelos participantes reais, preservando contratos.

7. Plano das três semanas

Período	Marco verificável	Prioridades
Semana 1	Abertura → diagnóstico → orçamento entre três serviços	Contratos, extração, AWS/IAM, filas/bancos e primeiro deploy
Semana 2	Reserva, pagamento, liberação e uma compensação completos	Mercado Pago de teste, DynamoDB, inbox/outbox, BDD e pipelines
Semana 3	Entrega estável e evidências revisadas	80% por serviço, Sonar, observabilidade, vídeo, PDF e links

Meta: primeira integração até o fim da semana 1, inicialmente com pagamento simulado. Começar testes e deploy desde a primeira semana. Confirmar horas disponíveis e acessos no início.

Reaproveitar `*Test`, `*IT` e `*CucumberTest`. Integração usa PostgreSQL e DynamoDB Local onde aplicável; uma execução AWS valida IAM, transações e SQS FIFO/DLQ. CI financeiro usa simulador, com evidência separada em conta Mercado Pago de teste. Adaptar tracing da fase 3 com correlação no envelope/atributos SQS e logs; CloudWatch sozinho não substitui trace distribuído.

8. Checklist completo dos entregáveis

Fonte: **SOAT - Fase 4 - Tech challenge.pdf**, páginas 2 a 6. As caixas são trabalho a comprovar. **M = Matheus, J = Julio, A = Anderson, R = Marcelo.**

8.1 Arquitetura e processo

- □ Três microsserviços independentes em repositórios próprios. **M/J/A + R**. Evidência: URLs, imagens e aplicações.
- □ Infraestrutura/configuração e banco próprios por serviço, sem acesso cruzado. **M/J/A + R**. Evidência: IaC, roles e recursos segregados.
- □ SQL e NoSQL com dados de negócio. **M/J/A**. Evidência: PostgreSQL e DynamoDB persistindo o atendimento.
- □ Estratégia REST e mensageria documentada. **Todos**. Evidência: OpenAPI, contratos SQS e diagrama.
- □ Integração Mercado Pago com registro/verificação de pagamentos. **J**. Evidência: teste de checkout, webhook/consulta e reflexo na ordem.
- □ Saga com rollback local e compensações nas falhas. **M/J/A**. Evidência: fluxo feliz, falha, liberação e estorno quando aplicável.
- □ Escolha de orquestração justificada nos READMEs. **M, revisão J/A/R**. Evidência: documentação dos contextos.
- □ Deploy automatizado Kubernetes dos três serviços. **R + donos**. Evidência: pipelines com rollout/smoke no EKS.

- □ Mensageria coordenando o processo. **R + donos**. Evidência: SQS com comandos/resultados reais.
- □ Observabilidade da fase 3 reutilizada/adaptada. **R + donos**. Evidência: dashboards, logs e traces.

8.2 Testes, qualidade e proteção

- □ Unitários nos três microsserviços. **M/J/A**. Evidência: relatórios de CI.
- □ Pelo menos um fluxo completo BDD. **M coordena; J/A fornecem steps; R fornece ambiente**. Evidência: Cucumber.
- □ Cobertura mínima de 80% em ordem. **M**. Evidência: JaCoCo/Sonar total e print/link no README.
- □ Cobertura mínima de 80% em billing. **J**. Evidência: JaCoCo/Sonar total e print/link no README.
- □ Cobertura mínima de 80% em execução. **A**. Evidência: JaCoCo/Sonar total e print/link no README.
- □ SonarQube ou similar no CI por serviço. **R configura; donos corrigem**. Evidência: análises/gates.
- □ Pipeline independente com build, testes, qualidade e deploy por serviço. **R + dono**. Evidência: três execuções aprovadas.
- □ Main protegida, PR obrigatório e checks automáticos. **R**. Evidência: regras e PR validado nos três repositórios.

8.3 Conteúdo obrigatório de cada repositório

Conferir cada caixa abaixo nos três serviços:

- □ Código-fonte. **Dono**. Evidência: revisão/tag da entrega.
- □ Dockerfile. **R fornece modelo; dono valida**. Evidência: imagem executável.
- □ Manifestos Kubernetes. **R + dono**. Evidência: arquivos versionados e deploy.
- □ Pipelines CI/CD. **R + dono**. Evidência: workflows e execuções.
- □ Evidência de cobertura no README por print ou link. **Dono**.
- □ Arquitetura do serviço documentada. **Dono, revisão R**. Evidência: banco, limites e comunicação.
- □ Swagger ou collection Postman atualizada. **Dono**. Evidência: endpoints e contratos acessíveis.

8.4 Vídeo obrigatório

- □ Até 15 minutos no YouTube ou Vimeo, público ou não listado. **Todos; R consolida publicação**.
- □ Demonstrar ordem completa pelos microsserviços. **M/J/A**.
- □ Demonstrar Saga, falha e compensação. **M conduz; J/A mostram efeitos**.
- □ Demonstrar deploy automatizado de pelo menos um serviço com testes validados. **R + dono**.
- □ Demonstrar monitoramento e rastreamento distribuído. **R + donos**.

8.5 PDF final do portal

- □ Nome e identificação dos participantes. **Todos; R consolida**.
- □ Links dos três repositórios. **Donos conferem; R consolida**.
- □ Link do vídeo até 15 minutos. **R; todos verificam acesso**.
- □ Diagrama geral final: serviços, bancos e comunicação. **R, revisão de todos**.
- □ Descrição da estratégia de Saga escolhida. **M, revisão J/A**.
- □ Justificativa da divisão e das tecnologias. **Todos**.
- □ PDF revisado e enviado por integrante designado. **Todos revisam**. Evidência: submissão no portal.

8.6 Aceites técnicos da proposta

- □ Repetição não duplica reserva, consumo ou cobrança; pod reiniciado retoma Saga.
- □ Última unidade e disputa entre autorização/cancelamento são protegidas.
- □ Timeout financeiro é conciliado; compensação incompleta gera pendência/alerta.
- □ Excluir SQS só após commit; testar DLQ, reprocessamento e sequência por Saga.
- □ IAM restringe filas/tabelas; secrets fora do Git; migrations/índices reproduzíveis.

Este PDF é o plano da equipe. O PDF final do portal receberá links e evidências reais após implementar.

9. Referências e anexos

- Anexo técnico: todas as transações e DynamoDB.
- Exemplo com endpoints atuais.
- Padrões Java e testes.
- SQS FIFO, visibility timeout e SqsClient Java.
- Transações DynamoDB, outbox AWS e EKS Pod Identity.
- Mercado Pago Checkout Pro.

10. Divisão de tarefas da equipe

10.1 Visão geral

Membro	Frente	Base atual
Matheus	Ordem e Saga	`ServiceOrderService`, acompanhamento, cadastros e testes
Julio	Orçamento e Mercado Pago	`ServiceOrderBudgetService`, aprovação, e-mail e testes
Anderson	Diagnóstico, catálogo, estoque e execução	`automotiveservice`, `inventory`, itens/insumos e testes
Marcelo	Plataforma AWS, segurança e automação	Docker, JWT, OpenAPI, Maven/Sonar e artefatos da fase 3

10.2 Matheus - Ordem de serviço e Saga

- □ Extrair ordem, cliente/veículo e acompanhamento para `numberone-service-order`, com RDS exclusivo; portar validações/testes.
- □ Retirar relacionamentos JPA com orçamento e itens executáveis externos; manter IDs/resumos.
- □ Criar migrations de Saga, histórico e inbox/outbox; implementar etapas, prazo global, recuperação e compensação.
- □ Consumir resultados da fila da ordem e produzir comandos financeiros/operacionais, validando versão/cancelamento.
- □ Atualizar pronto/iniciado/concluído/entregue; liderar BDD com Julio e Anderson.
- □ Entregar testes, 80% de cobertura, OpenAPI/README e evidências.

Aceite: fluxo e compensação retomáveis pelo PostgreSQL. **Primeiro passo:** publicar estados e comandos v1.

10.3 Julio - Orçamento, pagamento e Mercado Pago

- □ Extrair orçamento/aprovação/e-mail para `numberone-billing`, RDS próprio e sem escrita direta na ordem.
- □ Criar modelo financeiro, notificações, operações externas e inbox/outbox; preservar valores aprovados.
- □ Implementar checkout, webhook autenticado, consulta/conferência e correlação, com segredos no Secrets Manager.
- □ Implementar recusa, reconciliação, `CancelBilling`, estorno e pagamento tardio; confirmar compensação após concluir.
- □ Consumir comandos SQS e publicar resultados; validar duplicação e Mercado Pago em conta de teste.
- □ Entregar testes, 80% de cobertura, OpenAPI/README, steps BDD e evidências.

Aceite: finanças da ordem correta, sem duplicação. **Primeiro passo:** portar orçamento e fechar payloads de criação/cobrança.

10.4 Anderson - Estoque e execução

- □ Extrair diagnóstico, catálogo, itens/insumos e estoque; reaproveitar regras Java e trocar adaptadores JPA por AWS SDK DynamoDB.
- □ Definir chaves/índices e importação de dados fictícios de trabalho, catálogo, saldo, reservas, movimentos e inbox/outbox.
- □ Implementar diagnóstico/resultados e reserva condicional com `TransactWriteItems`, sem efeitos parciais.
- □ Implementar autorização/cancelamento versionados, consumo único e conclusão, sempre com outbox.
- □ Consumir SQS e publicar resultados; testar última unidade, repetição, início concorrente e liberação.
- □ Entregar testes DynamoDB Local/AWS, 80% de cobertura, OpenAPI/README, steps BDD e evidências.

Aceite: estoque consistente e compensação verificável. **Primeiro passo:** validar chaves/consultas e reserva atômica antes de portar todos os endpoints.

10.5 Marcelo - Plataforma e integração

- □ Publicar `numberone-platform` com IaC, outputs e convenções; confirmar/reaproveitar EKS/observabilidade da fase 3.
- □ Entregar SQS FIFO/DLQs, tabelas-base DynamoDB e RDS segregados; IAM/Pod Identity, secrets e guias de acesso.
- □ Entregar aplicação mínima AWS SDK v2 com fixtures, reentrega/DLQ, visibility timeout e exclusão após commit.
- □ Publicar templates Docker, Kubernetes/Ingress, OIDC/ECR, CI/Sonar/cobertura e deploy EKS; validar com aplicação mínima e proteger main.
- □ Disponibilizar logs/alarmes CloudWatch e integração de trace da fase 3, com correlação e runbook de recuperação.
- □ Entregar ambiente BDD/smoke e modelo de evidências, diagrama/PDF e vídeo; consolidar resultados integrados.

Entregas independentes: IaC, convenções, fixtures e templates funcionais. Donos adotam padrões e implementam regras. **Primeiro passo:** filas/IAM e ambiente de desenvolvimento, em paralelo aos acessos do cluster/CI.

10.6 Acordos de execução

Matheus decide a sequência; Julio implementa finanças; Anderson implementa estoque/execução; Marcelo publica plataforma e guias. Cada dono entrega persistência, mensagens, testes e documentação do serviço. Revisar contratos primeiro e usar fixtures para desenvolver em paralelo. Na semana 3, os quatro validam checklist, links e evidências.